



Техническое задание (ТЗ): Создание ИИ-Агента через FastMCP

Цель воркшопа: Научиться создавать кастомные инструменты (tools) для LLM с помощью протокола MCP (Model Context Protocol) и подключать готовые MCP-серверы. В результате мы создадим автономного ИИ-агента, который умеет узнавать погоду через API, управлять браузером через Playwright MCP, и сохранять заметки на ваш компьютер.

Стек технологий: Python, `fastmcp`, `httpx`, Node.js (для Playwright MCP).



Подготовка к работе

Python-зависимости

```
pip install fastmcp httpx
```

Playwright MCP (готовый сервер от Microsoft)

Для Задания 2 нам потребуется готовый MCP-сервер Playwright MCP. Он не требует отдельной установки — запускается через `npx`.

Требования: Node.js 18+ ([скачать](#))

Проверьте, что Node.js установлен:

```
node --version
```



Задание 1: Инструмент «Умная погода» (API)

Описание: Научить ИИ узнавать текущую температуру на улице, чтобы он мог давать адекватные советы (гулять в парке или идти в кино).

Что нужно сделать:

1. Создать файл `weather_server.py` с сервером `FastMCP("WeatherAgent")`.
2. Написать функцию `get_weather(city_name: str)`, которая принимает название города.
3. Использовать библиотеку `httpx` для отправки GET-запросов к бесплатному API **Open-Meteo**.
4. Оборнуть функцию декоратором `@mcp.tool()`.

API для использования:

1) **Geocoding API** — конвертация названия города в координаты:

```
GET <https://geocoding-api.open-meteo.com/v1/search?name={city_name}&count=1&language=ru&format=json>
```

Пример ответа:

```
{
  "results": [
    {
      "name": "Almaty",
      "latitude": 43.25,
      "longitude": 76.91667,
      "country": "Kazakhstan",
      "timezone": "Asia/Almaty"
    }
  ]
}
```

Нужные поля: `results[0]["latitude"]` и `results[0]["longitude"]`.

2) **Forecast API** — получение текущей погоды по координатам:

```
GET <https://api.open-meteo.com/v1/forecast?latitude={lat}&longitude={lon}&current=temperature_2m,wind_speed_10m,weather_c
```

```
ode,is_day>
```

Пример ответа:

```
{
  "latitude": 43.25,
  "longitude": 77.0,
  "elevation": 834.0,
  "current_units": {
    "temperature_2m": "°C",
    "wind_speed_10m": "km/h"
  },
  "current": {
    "time": "2026-02-26T12:00",
    "temperature_2m": 1.8,
    "wind_speed_10m": 9.6,
    "weather_code": 61,
    "is_day": 1
  }
}
```

Нужные поля: `current["temperature_2m"]`, `current["wind_speed_10m"]`,
`current["weather_code"]`.

Подсказка: Параметр `current=` принимает список переменных через запятую. Полный список: `temperature_2m`, `relative_humidity_2m`, `apparent_temperature`, `precipitation`, `rain`, `weather_code`, `wind_speed_10m`, `wind_direction_10m`, `is_day` и другие.

Коды погоды (weather_code): 0 — ясно, 1-3 — облачно, 45-48 — туман, 51-55 — морось, 61-65 — дождь, 71-75 — снег, 80-82 — ливень, 95-99 — гроза. Можно сделать словарь для перевода кодов в текст.

Критерий приёмки:

При запросе к ИИ *"Какая сейчас погода в Алматы?"*, агент вызывает инструмент и отвечает точной температурой в °C, скоростью ветра и

описанием погоды.

Задание 2: Инструмент «Афиша» (Playwright MCP)

Описание: Дать ИИ "глаза" в интернете. Вместо написания кода для парсинга вручную, мы подключим готовый MCP-сервер **Playwright MCP** от Microsoft, который даёт ИИ полноценное управление браузером.

Что нужно сделать:

1. **Подключить Playwright MCP сервер** к вашему MCP-клиенту (Claude Desktop, Cursor или VS Code).
2. **Отправить ИИ запрос**, чтобы он сам открыл сайт с афишей и собрал информацию.

Настройка Playwright MCP:

Для Claude Desktop

Откройте файл настроек (**Settings** → **Developer** → **Edit Config**) и добавьте:

```
{
  "mcpServers": {
    "playwright": {
      "command": "npx",
      "args": ["@playwright/mcp@latest"]
    }
  }
}
```

Для Cursor

Cursor Settings → **MCP** → **Add new MCP Server** :

- Name: **playwright**
- Type: **command**

- Command: `npx @playwright/mcp@latest`

Для VS Code

Выполните в терминале:

```
code --add-mcp '{"name":"playwright","command":"npx","args":["@playwright/mcp@latest"]}'
```

Какие инструменты даёт Playwright MCP:

Инструмент	Описание
<code>browser_navigate</code>	Перейти по URL
<code>browser_snapshot</code>	Получить структуру страницы (accessibility tree)
<code>browser_click</code>	Кликнуть на элемент
<code>browser_type</code>	Ввести текст в поле
<code>browser_press_key</code>	Нажать клавишу (Enter, Tab и т.д.)
<code>browser_wait_for</code>	Подождать появления текста
<code>browser_take_screenshot</code>	Сделать скриншот

Ключевая идея: ИИ использует `browser_snapshot` чтобы "прочитать" страницу в виде структурированного дерева элементов, а затем сам решает, куда кликнуть и что ввести. Вам не нужно писать CSS-селекторы — ИИ разбирается сам!

Как проверить:

Отправьте ИИ промпт:

"Открой сайт <https://sxodim.com/almaty> с помощью `browser_navigate`, затем сделай `browser_snapshot` и найди 5 ближайших мероприятий в Алматы. Верни мне список с названиями и датами."

Критерий приёмки:

Агент самостоятельно открывает браузер, переходит на сайт, читает содержимое страницы через `snapshot` и возвращает список реальных

мероприятий.

Задание 3: Инструмент «Менеджер заметок» (Локальная файловая система)

Описание: Дать ИИ долгосрочную память и возможность физически создавать файлы на вашем компьютере.

Что нужно сделать:

1. В том же файле `weather_server.py` (или отдельном `notes_server.py`) добавить инструменты для работы с заметками.
2. Написать функцию `save_note(filename: str, content: str)`, которая создаёт `.txt` файл в папке `my_notes` с переданным названием и записывает туда контент.
3. Написать функцию `list_notes()`, которая читает папку `my_notes` и возвращает список названий всех сохранённых файлов.
4. Написать функцию `read_note(filename: str)`, которая читает содержимое указанного файла из папки `my_notes`.
5. Оборнуть все функции декоратором `@mcp.tool()`.

Подсказка: Используйте `os.makedirs("my_notes", exist_ok=True)` для создания папки (если она ещё не существует) и `pathlib.Path` или `os.path` для безопасной работы с путями.

Критерий приёмки:

Если попросить ИИ *"Сохрани рецепт пиццы в файл pizza"*, в папке `my_notes` появится реальный файл `pizza.txt` с рецептом. Если попросить *"Какие у меня есть заметки?"*, он вернёт список файлов.

Задание 4: Сборка Агента-Планировщика

Описание: Финальный босс. Вы написали кастомные инструменты (погода, заметки) и подключили Playwright MCP. Теперь нужно заставить ИИ использовать их в связке для решения сложной задачи.

Что нужно сделать:

1. Убедиться, что ваш MCP-сервер с инструментами погоды и заметок запущен и подключён к клиенту.
2. Убедиться, что Playwright MCP сервер тоже подключён.
3. **Отправить ИИ следующий промпт:**

"Я живу в Алматы. Спланируй мне сегодняшний вечер. Сначала проверь погоду с помощью своего инструмента `get_weather`. Если температура ниже 10 градусов, зайди на сайт <https://sxodim.com/almaty> через Playwright и найди мероприятия в помещениях (театры, стендапы, концерты). Если тепло — предложи прогулку на улице. Составь красивое расписание на вечер по часам и сохрани его в файл `'my_perfect_evening'` с помощью `save_note`."

Критерий приёмки:

Агент самостоятельно, без ваших подсказок:

1. Вызывает `get_weather("Алматы")` — получает температуру.
2. В зависимости от температуры — использует Playwright для поиска мероприятий на сайте.
3. Генерирует текстовый план вечера.
4. Вызывает `save_note("my_perfect_evening", ...)` — сохраняет файл.
5. Вы находите готовый план в папке `my_notes/my_perfect_evening.txt`.

БОНУСНОЕ ЗАДАНИЕ: Поиск в 2GIS (Playwright MCP)

Описание: Научить агента искать конкретные заведения на карте города, используя Playwright MCP.

Что нужно сделать:

Отправьте ИИ промпт, который заставит его:

1. Открыть `https://2gis.kz/almaty` через `browser_navigate`.

2. Сделать `browser_snapshot`, чтобы найти поле поиска.
3. Ввести запрос (например, "кофейни") через `browser_type`.
4. Нажать Enter через `browser_press_key`.
5. Подождать загрузки результатов через `browser_wait_for`.
6. Сделать `browser_snapshot` и прочитать первые 3-5 результатов.

Пример промпта:

"Зайди на 2gis.kz/almaty, найди 5 ближайших кофеен через поиск на сайте. Для каждой верни название и адрес."

Почему это интересно:

2GIS — реальный продакшн-сайт с динамическим контентом. ИИ с Playwright MCP должен сам разобраться в структуре страницы через accessibility snapshot и выполнить все действия. Это демонстрирует силу подхода — вам не нужно писать ни строчки парсинга!

Критерий приёмки:

Агент возвращает реальные названия и адреса заведений из 2GIS.